

Submission Worksheet

Submission Data

Course: IT202-010-S2026

Assignment: IT202 - Milestone 3

Student: Joshua T. (jlt36)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 5/7/2026 1:49:13 AM

Updated: 5/7/2026 9:45:26 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-3/grading/jlt36>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-3/view/jlt36>

Instructions

1. Refer to Milestone3 of this doc: <https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone3 branch
 1. `git checkout Milestone3` (ensure proper starting branch)
 2. `git pull origin Milestone3` (ensure history is up to date)
5. Fill out the worksheet below
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to GitHub
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone3`
 4. On GitHub merge the pull request from Milestone3 to qa
 5. On GitHub create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the Render.com prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout qa`
11. `git pull origin qa`

Section #1: (3 pts.) Api Data

Progress: 100%

⇒ Task #1 (1 pt.) - Concept of Data Association

Progress: 100%

Details:

- What is the concept of your data association to users? (examples: favorites, wish list, purchases, assignment, etc)
- Describe with a few sentences

Your Response:

The concept of my data association to users will be saved and favorited memes. Users will be able to save memes from the reddit meme api to their account so they can view them later. This creates a relationship between a logged in user and a meme without changing the actual meme data itself. For example, one user can save a meme to their personal list while another user may save a different set of memes.



Saved: 5/7/2026 1:50:45 AM

⇒ Task #2 (1 pt.) - Data Updates

Progress: 100%

Details:

- When an associated entity is updated (manually or via API), how is the association affected?
 - Does the user see the old version of the data?
 - Does the user see the new version of the data?
 - Does the user need to have data re-associated or remapped?
- Explain why.

Your Response:

When a saved meme gets updated manually or from the API, the user will see the new updated version of that meme. The saved/fav table only connects the user to the meme using something like user_id and meme_id. I doesn't save a separate copy of the meme. The user does not need to save the meme again or have anything remapped. Their saved meme will still point to the same meme record so if the title, image, etc changes, the saved lists will automatically show the updated version.



Saved: 5/7/2026 4:21:32 AM

≡ Task #3 (1 pt.) - Handling Association

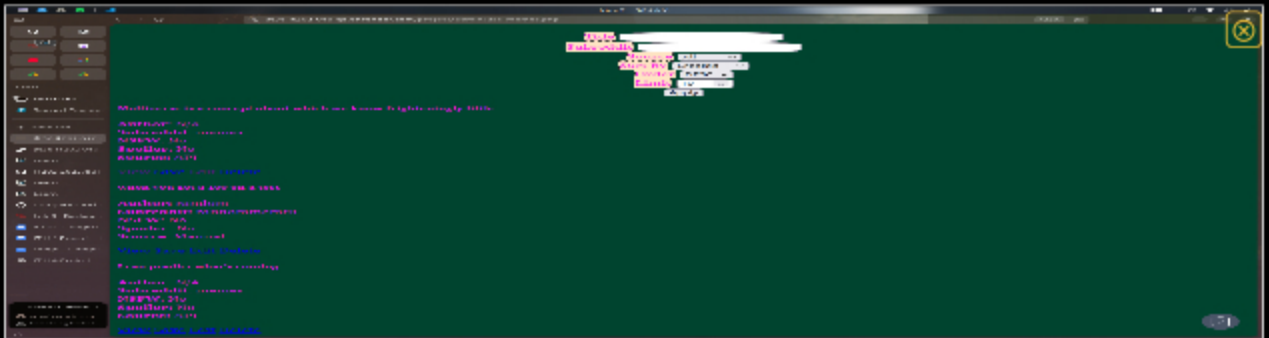
Progress: 100%

🖼 Part 1:

Progress: 100%

Details:

- Show an example page of where a user can get data associated with them
- Ensure the Render.com qa URL is visible
- Caption if this is a user-facing page or admin page
- Include screenshots of code related to this logic including the database query



Admin facing page where an admin can save a meme to their account.

```
39 // UCID: jlt36
40 // Date: 05/07/2026
41 // Description: Inserts or reactivates the relationship between the logged in user and selected meme
42
43 $query = "INSERT INTO `IT202-M25-UserMemes`
44         (`user_id`, `meme_id`, `is_active`)
45         VALUES
46         (:user_id, :meme_id, 1)
47         ON DUPLICATE KEY UPDATE
48         is_active = 1,
49         modified = CURRENT_TIMESTAMP";
50
51 try {
52     $stmt = $db->prepare($query);
53     $stmt->execute([
54         ":user_id" => $user_id,
55         ":meme_id" => $meme_id
56     ]);
57 }
```

code for database query

```
<!-- UCID: jlt36 -->
<!-- Date: 05/07/2026 -->
<!-- Description: save button -->

<form method="POST" action="<?php echo get url("save_meme.php"); ?>" style="display:inline;>
    <input type="hidden" name="meme_id" value="<?php se($record, "id"); ?>">
    <button type="submit" style="background:none; border:none; color:#0000EE; text-decoration:underline; cursor:pointer; padding:0; font:inherit;">
        Save
    </button>
</form>
```

save button



Saved: 5/7/2026 5:36:29 AM

Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page that triggers and handles this logic

URL #1
<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/48>



URL
<https://github.com/Joshua-Emma>



URL #2
https://jlt36-it202-010-qa.onrender.com/project/admin/list_memes.php



URL
<https://jlt36-it202-010-qa.onrende>



Saved: 5/7/2026 5:36:29 AM

≡ Part 3:

Progress: 100%

Details:

- Describe the process of associating data with the user.
- Can it be toggled, or is it applied once?

Your Response:

The user can save a meme by clicking the Save Meme button on the meme list page. When they click it, the meme's id gets sent to save_meme.php. From there, the page uses the logged in user's id and the selected meme's id to add a record into the UserMeme table. If a user already saved that meme before, it shouldn't make a duplicate because the table connects one user to one meme only once. If the saved meme was removed or marked inactive before, it can be added back again. So the user saves a meme, keeps it in their list, and can remove it later if they want.

Saved: 5/7/2026 5:36:29 AM

Section #2: (6 pts.) Associations

Progress: 100%

≡ Task #1 (1.50 pts.) - Logged-in User's Associated Entities

Progress: 100%

Details:

- Each line item should have a logical summary
- Each line item should have a link/button to a single view page of the entity
- Each line item should have a link/button to a delete action of the relationship (doesn't delete the entity or user, just the relationship)
- The page should have a link/button to remove all associations for the particular user
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
 - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)

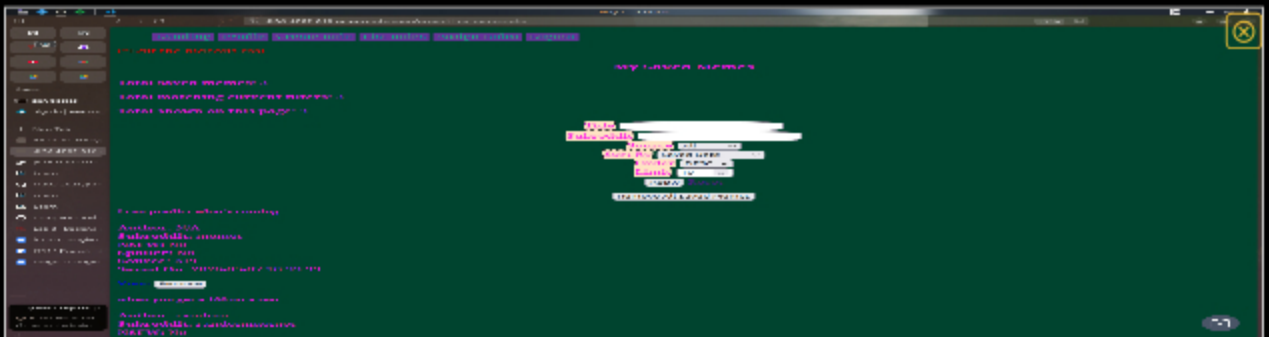
- enforces rules/
 - o A filter with no matching records should show "no results available" or equivalent

Part 1:

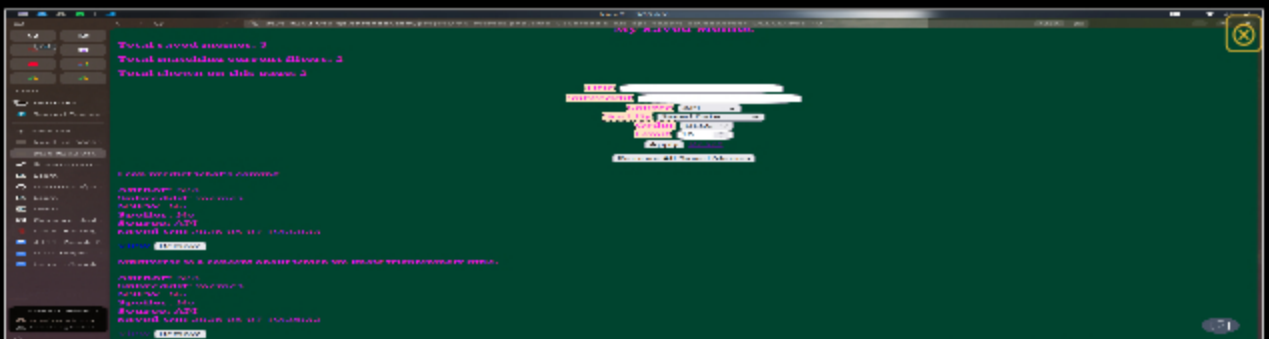
Progress: 100%

Details:

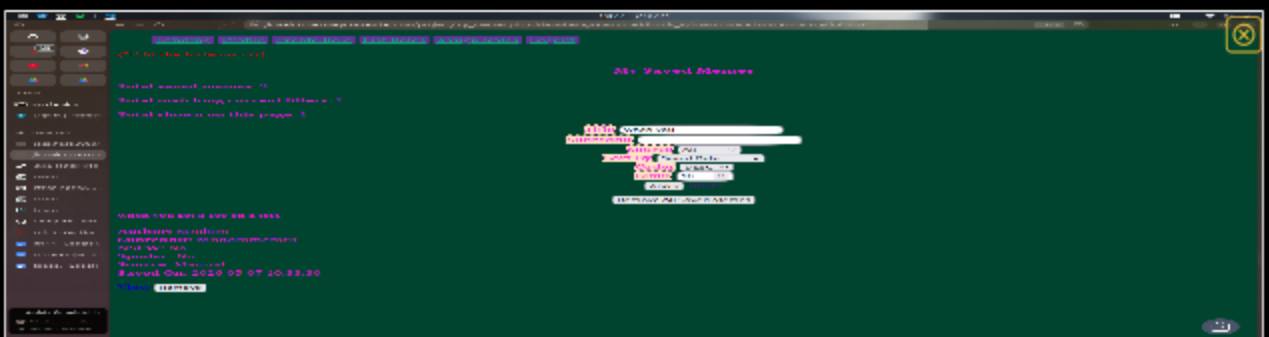
- Show a few examples of this page from Render.com qa with various filters applied
- Ensure the Render.com qa URL is visible
- Ensure each requirement is visible
- Include the code screenshots related to this page



normal my_memes view



api only filter



filter by title

```
// UCID: jlt36
// Date: 05/07/2020
// Description: checks for logged in user and current user id + this code handles the filter/sort options and enforces the record limit between 1 and 100.
```

```
is_logged_in(true);
```

```
$user_id = get_user_id();
```

```
if (slimit < 1 || slimit > 100) {
    slimit = 10;
}
```

filter/sort/limit/check user logged in code

count queries

join query

```
<form method="POST" action="<?php echo get_url('remove_all_my_memes.php'); ?>" onsubmit="return confirm('Remove all saved memes from your account?');" class="mb-4">
  <button type="submit" class="btn btn-danger">Remove All Saved Memes</button>
</form>

<?php if (count($results) == 0) { ?>
```

remove all button

```
148 <div class="container-fluid">
149     <h3>My Saved Memes</h3>      You, 1 hour ago • Add saved meme list and removal actions
150
151     <div class="mb-3">
152         <p><strong>Total saved memes:</strong> <?php se($total_saved); ?></p>
153         <p><strong>Total matching current filters:</strong> <?php se($filtered_total); ?></p>
154         <p><strong>Total shown on this page:</strong> <?php echo count($results); ?></p>
155     </div>
156
157     <form method="GET" class="row g-3 mb-4">
```

stats display

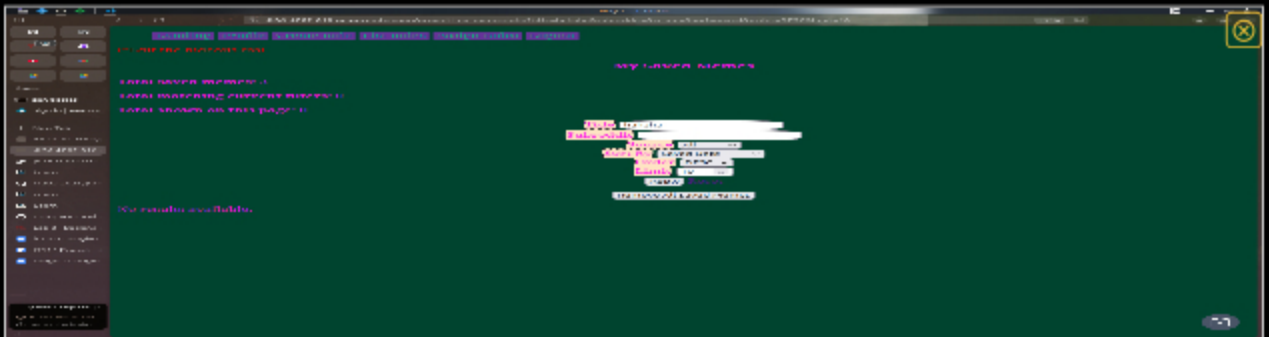
```
// description: Each word here shows a summary, a view button for the single word page,
```



```
// and a Remove button that removes only the saved information.
// If the remove button has been clicked, the results available.

Table of Results (multi) - 01:17
1. Remove results from table.
2. Table view.
3. Table of Results (multi) - 01:17
4. Table of Results (multi) - 01:17
5. Table of Results (multi) - 01:17
6. Table of Results (multi) - 01:17
7. Table of Results (multi) - 01:17
8. Table of Results (multi) - 01:17
9. Table of Results (multi) - 01:17
10. Table of Results (multi) - 01:17
11. Table of Results (multi) - 01:17
12. Table of Results (multi) - 01:17
13. Table of Results (multi) - 01:17
14. Table of Results (multi) - 01:17
15. Table of Results (multi) - 01:17
16. Table of Results (multi) - 01:17
17. Table of Results (multi) - 01:17
18. Table of Results (multi) - 01:17
19. Table of Results (multi) - 01:17
20. Table of Results (multi) - 01:17
21. Table of Results (multi) - 01:17
22. Table of Results (multi) - 01:17
23. Table of Results (multi) - 01:17
24. Table of Results (multi) - 01:17
25. Table of Results (multi) - 01:17
26. Table of Results (multi) - 01:17
27. Table of Results (multi) - 01:17
28. Table of Results (multi) - 01:17
29. Table of Results (multi) - 01:17
30. Table of Results (multi) - 01:17
31. Table of Results (multi) - 01:17
32. Table of Results (multi) - 01:17
33. Table of Results (multi) - 01:17
34. Table of Results (multi) - 01:17
35. Table of Results (multi) - 01:17
36. Table of Results (multi) - 01:17
37. Table of Results (multi) - 01:17
38. Table of Results (multi) - 01:17
39. Table of Results (multi) - 01:17
40. Table of Results (multi) - 01:17
41. Table of Results (multi) - 01:17
42. Table of Results (multi) - 01:17
43. Table of Results (multi) - 01:17
44. Table of Results (multi) - 01:17
45. Table of Results (multi) - 01:17
46. Table of Results (multi) - 01:17
47. Table of Results (multi) - 01:17
48. Table of Results (multi) - 01:17
49. Table of Results (multi) - 01:17
50. Table of Results (multi) - 01:17
51. Table of Results (multi) - 01:17
52. Table of Results (multi) - 01:17
53. Table of Results (multi) - 01:17
54. Table of Results (multi) - 01:17
55. Table of Results (multi) - 01:17
56. Table of Results (multi) - 01:17
57. Table of Results (multi) - 01:17
58. Table of Results (multi) - 01:17
59. Table of Results (multi) - 01:17
60. Table of Results (multi) - 01:17
61. Table of Results (multi) - 01:17
62. Table of Results (multi) - 01:17
63. Table of Results (multi) - 01:17
64. Table of Results (multi) - 01:17
65. Table of Results (multi) - 01:17
66. Table of Results (multi) - 01:17
67. Table of Results (multi) - 01:17
68. Table of Results (multi) - 01:17
69. Table of Results (multi) - 01:17
70. Table of Results (multi) - 01:17
71. Table of Results (multi) - 01:17
72. Table of Results (multi) - 01:17
73. Table of Results (multi) - 01:17
74. Table of Results (multi) - 01:17
75. Table of Results (multi) - 01:17
76. Table of Results (multi) - 01:17
77. Table of Results (multi) - 01:17
78. Table of Results (multi) - 01:17
79. Table of Results (multi) - 01:17
80. Table of Results (multi) - 01:17
81. Table of Results (multi) - 01:17
82. Table of Results (multi) - 01:17
83. Table of Results (multi) - 01:17
84. Table of Results (multi) - 01:17
85. Table of Results (multi) - 01:17
86. Table of Results (multi) - 01:17
87. Table of Results (multi) - 01:17
88. Table of Results (multi) - 01:17
89. Table of Results (multi) - 01:17
90. Table of Results (multi) - 01:17
91. Table of Results (multi) - 01:17
92. Table of Results (multi) - 01:17
93. Table of Results (multi) - 01:17
94. Table of Results (multi) - 01:17
95. Table of Results (multi) - 01:17
96. Table of Results (multi) - 01:17
97. Table of Results (multi) - 01:17
98. Table of Results (multi) - 01:17
99. Table of Results (multi) - 01:17
100. Table of Results (multi) - 01:17
```

Line item summary + View + Remove + No results



no results filter

 Saved: 5/7/2026 7:23:33 AM

Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page

URL #1
<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/50>

URL #2
https://jlt36-it202-010-qa.onrender.com/project/my_memes.php



URL
<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/50>



URL
https://jlt36-it202-010-qa.onrender.com/project/my_memes.php



 Saved: 5/7/2026 7:23:33 AM

Part 3:

Progress: 100%

Details:

- Describe how you implemented the association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

I made the association output on the my_memes.php page. This page shows only the memes saved by the currently logged-in user. It gets the user's id, checks the UserMemes table, and joins it with the Memes table so the saved meme details can be shown.

Each saved meme has a short summary with the title, author, subreddit, source, and saved date. Each item also has a View button to go to the meme's details page and a Remove button that only deletes the saved relationship, not the actual meme.

I also added stats at the top showing the total saved memes, how many match the current filters, and how many are shown on the page. The page has filters for title, subreddit, source, sorting, order, and a limit field. The limit is checked on the server side so it stays between 1 and 100. If nothing matches the filter, the page shows "no results available" There is also a remove all saved memes button to remove all saved meme relationships for that user.



Saved: 5/7/2026 7:23:33 AM

☰ Task #2 (1.50 pts.) - All Users Association Page

Progress: 100%

Details:

- Each line item should have a logical summary
- Each line item should include the username this entity is associated with
 - Clicking the username should redirect to that user's public profile
- Each line item should include a column that shows the total number of users the entity is associated with
- Each line item should have a link/button to a single view page of the entity
- Each line item should have a link/button to a delete action of the relationship (doesn't delete the entity or user, just the relationship)
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
 - Include a filter for partial username match to show only matching users' associated entities
 - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
 - A filter with no matching records should show "no results available" or equivalent
- When a filter is applied, the page should have a link/button to remove all associations for the matching user(s)

🖼 Part 1:

Progress: 100%

Details:

- [illegible]

[illegible]

```
// UCID: jlt36
// Date: 05/07/2026
// Description: Handles filters, sorting, and limit for all user meme associations
$username = trim($_GET["username"] ?? "");
$title = trim($_GET["title"] ?? "");
$subreddit = trim($_GET["subreddit"] ?? "");
$sort = $_GET["sort"] ?? "saved";
$order = strtoupper($_GET["order"] ?? "DESC");
$limit = (int)($_GET["limit"] ?? 10);

if ($limit < 1 || $limit > 100) {
    $limit = 10;
}
```

```
// M3AD: jlt30
// Date: 02/07/2020
// Description: Counts all active user name associations and the amount matching the current filters
select count(*) as M3AD_0;
```

```

$filtered_total = 0;

try {
    $stat = $db->prepare("SELECT COUNT(*) AS total FROM `IT202-M25-UserMemes` WHERE is_active = 1");
    $stmt->execute();
    $total_associations = (int)($stmt->fetch()["total"] / 10);

    $count_query = "SELECT COUNT(*) AS total
    FROM `IT202-M25-UserMemes` um
    JOIN Users u ON um.user_id = u.id
    JOIN `IT202-M25-Memes` m ON um.meme_id = m.id
    WHERE sql";

    $stat = $db->prepare($count_query);
    foreach ($params as $key => $value) {
        $stmt->bindValue($key, $value);
    }
    $stmt->execute();
    $filtered_total = (int)($stmt->fetch()["total"] / 10);
} catch (PDOException $e) {
    error_log("Error counting all user meme associations: " . var_export($e, true));
    flash("There was a problem loading association stats.", "danger");
}

```

stats queries

```

// UCID: jlt36
// Date: 05/07/2026
// Description: Gets all user meme associations with usernames, meme info, and total users associated with each meme.
$query = "SELECT
    um.id AS association_id,
    um.created AS saved_at,
    u.id AS user_id,
    u.username,
    m.id AS meme_id,
    m.title,
    m.author,
    m.subreddit,
    m.is_nsfw,
    m.spiller,
    (
        SELECT COUNT(*)
        FROM `IT202-M25-UserMemes` um2
        WHERE um2.meme_id = m.id
        AND um2.is_active = 1
    ) AS user_count
FROM `IT202-M25-UserMemes` um
JOIN Users u ON um.user_id = u.id
JOIN `IT202-M25-Memes` m ON um.meme_id = m.id
WHERE sql
ORDER BY ` ` . $allowed_sorts[$sort] . ` ` ORDER
LIMIT :limit";

$results = [];

```

JOIN query

```

// UCID: jlt36
// Date: 05/07/2026
// Description: Soft removes all meme associations for users matching the username filter
$query = "UPDATE `IT202-M25-UserMemes` um
    JOIN Users u ON um.user_id = u.id
    SET um.is_active = 0, um.modified = CURRENT_TIMESTAMP
    WHERE u.username LIKE :username
    AND um.is_active = 1";

// UCID: jlt36
// Date: 05/07/2026
// Description: Soft removes one user meme association without deleting the user or meme
$query = "UPDATE `IT202-M25-UserMemes`
    SET is_active = 0, modified = CURRENT_TIMESTAMP
    WHERE id = :association_id
    LIMIT 1";

```

table output and remove matching users button

```

// UCID: jlt36
// Date: 05/07/2026
// Description: Soft removes all meme associations for users matching the username filter
$query = "UPDATE `IT202-M25-UserMemes` um
    JOIN Users u ON um.user_id = u.id
    SET um.is_active = 0, um.modified = CURRENT_TIMESTAMP
    WHERE u.username LIKE :username
    AND um.is_active = 1";

```

remove_matching_user_memes

```

// UCID: jlt36
// Date: 05/07/2026
// Description: Soft removes one user meme association without deleting the user or meme
$query = "UPDATE `IT202-M25-UserMemes`
    SET is_active = 0, modified = CURRENT_TIMESTAMP
    WHERE id = :association_id
    LIMIT 1";

```



Saved: 5/7/2026 8:14:07 AM

Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page

URL #1

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/52>



URL

<https://github.com/Joshua-Emma>



URL #2

https://jlt36-it202-010-qa.onrender.com/project/admin/all_user_memes.php



URL

<https://jlt36-it202-010-qa.onrende>



Saved: 5/7/2026 8:14:07 AM

Part 3:

Progress: 100%

Details:

- Describe how you implemented the association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

I made this page as an admin-facing page that shows all saved meme associations across users. It joins the UserMemes, Users, and Memes tables so the admin can see which user saved which meme.

Each row shows the username, meme summary, total users connected to that meme, saved date, and action buttons. The username links to the user's profile, view goes to the meme details page, and remove only deletes the relationship, not the user or meme.

I also added stats for total associations, matching results, and results shown. The page has filters for username, title, subreddit, sorting, order, and limit. The username filter works with partial matches, and the limit is checked on the server side so it stays between 1 and 100. If nothing matches, it shows "no results available." When a username filter is applied, a button appears to remove associations for the matching users.



Saved: 5/7/2026 8:14:07 AM

no results + meme card output



stats + filter form

 Saved: 5/7/2026 8:53:00 PM

↩ Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page

URL #1

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/53>



URL

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/53>



URL #2

https://jlt36-it202-010-qa.onrender.com/project/admin/unassociated_memes.php



URL

https://jlt36-it202-010-qa.onrender.com/project/admin/unassociated_memes.php



 Saved: 5/7/2026 8:53:00 PM

⇒ Part 3:

Progress: 100%

Details:

- Describe how you implemented the association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

I implemented this as an admin facing page that shows memes that are not currently saved or associated with any user. The page uses a LEFT JOIN between the Memes table and the UserMemes association table, then filters for records where there is no active association. Each meme is shown as a card with a short summary, including the title, author, subreddit, source, and created date. Each card also has a view button that links to the single meme details page. I added a stats section that shows the total unassociated memes, how many match the current filters and

how many are shown on the page. The page also has filters for title, subreddit, source, sorting, order, and limit. The limit is checked on the server side so it stays between 1 and 100. If no memes match the filter, the page shows "no results available."



Saved: 5/7/2026 8:53:00 PM

≡ Task #4 (1.50 pts.) - Admin Association Page (Like User Roles)

Progress: 100%

Details:

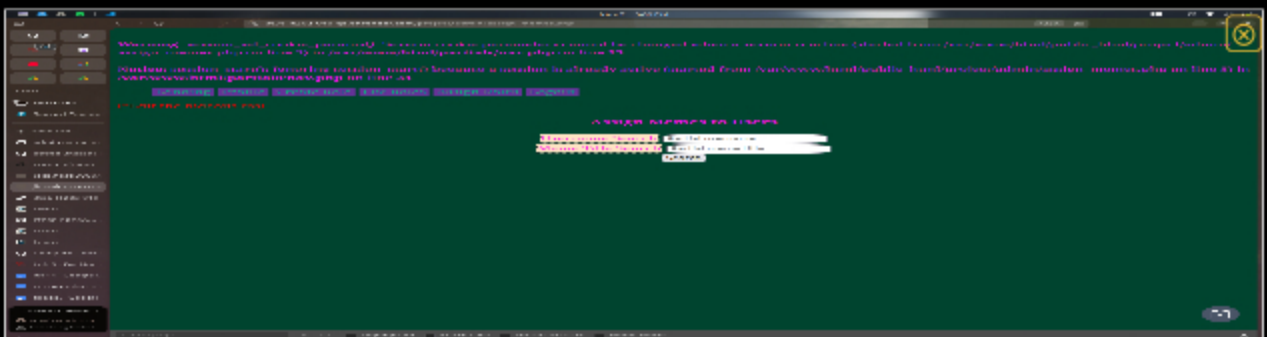
- The page should have a form with two fields
 - Partial match for username
 - Partial match for entity reference (name or something user-friendly)
- Submitting the form should give up to 25 matches of each
 - Likely best to show as two separate columns
- Each entity and user will have a checkbox next to them
- Submitting the checked associations should apply the association if it doesn't exist; otherwise it should remove the association
 - A filter with no matching records should show "no results available" or equivalent

📁 Part 1:

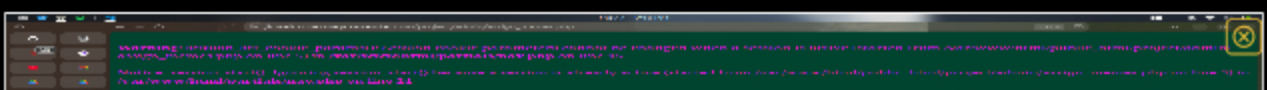
Progress: 100%

Details:

- Show a few examples of this page from Render.com qa with various selections having been submitted
- Ensure the Render.com qa URL is visible
- Ensure each requirement is visible
- Include screenshots of code related to this logic including the database query

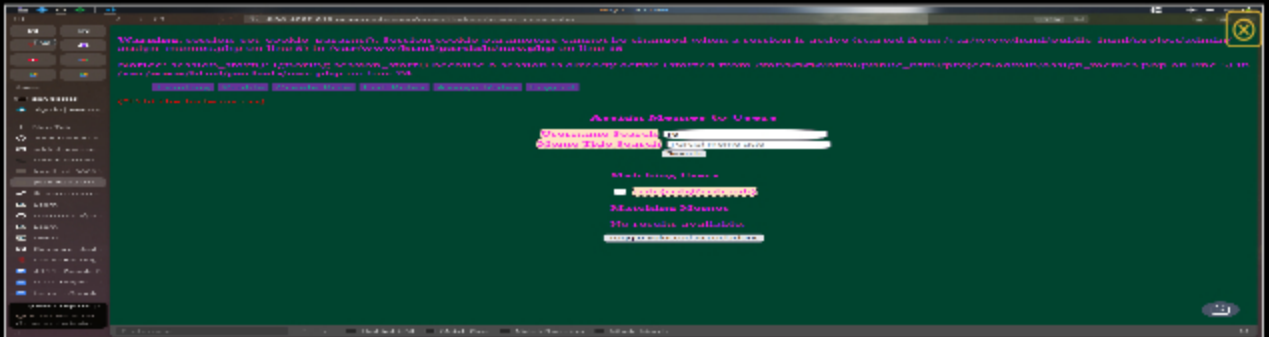


normal meme assign page





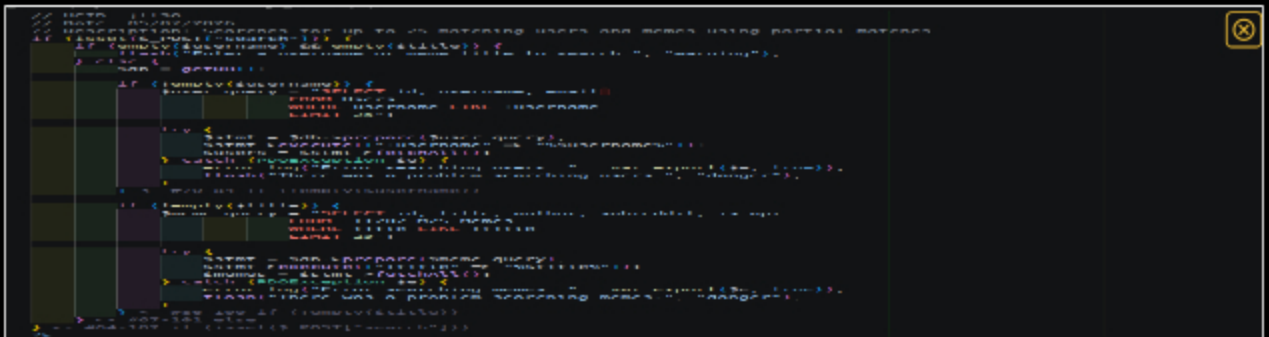
partial match for meme title filter



partial match for username filter



toggle association query



search queries



```

<div class="form">
  <div class="row">
    <div class="col">
      <input type="text" value="" />
    </div>
    <div class="col">
      <input type="text" value="" />
    </div>
  </div>
  <div class="row">
    <div class="col">
      <input type="button" value="Search" />
    </div>
  </div>
</div>

<div class="row">
  <div class="col">
    <table border="1">
      <thead>
        <tr>
          <th>User</th>
          <th>Meme</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>John</td>
          <td>Meme 1</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 2</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 3</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 4</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 5</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 6</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 7</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 8</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 9</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 10</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 11</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 12</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 13</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 14</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 15</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 16</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 17</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 18</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 19</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 20</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 21</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 22</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 23</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 24</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 25</td>
        </tr>
      </tbody>
    </table>
  </div>
  <div class="col">
    <table border="1">
      <thead>
        <tr>
          <th>User</th>
          <th>Meme</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>John</td>
          <td>Meme 1</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 2</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 3</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 4</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 5</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 6</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 7</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 8</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 9</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 10</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 11</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 12</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 13</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 14</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 15</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 16</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 17</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 18</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 19</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 20</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 21</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 22</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 23</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 24</td>
        </tr>
        <tr>
          <td>John</td>
          <td>Meme 25</td>
        </tr>
      </tbody>
    </table>
  </div>
</div>

```

checkbox output and no results msg



Saved: 5/7/2026 9:26:07 PM

Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page

URL #1

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/54>


URL

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/54>


URL #2

https://jlt36-it202-010-qa.onrender.com/project/admin/assign_memes.php


URL

https://jlt36-it202-010-qa.onrender.com/project/admin/assign_memes.php


Saved: 5/7/2026 9:26:07 PM

Part 3:

Progress: 100%

Details:

- Describe how your code solves the requirements; be clear

Your Response:

I made this as an admin-facing page where an admin can assign memes to users. The page has two search fields. One for username and one for meme title. Both use partial matching and only show up to 25 results.

The results show in two separate columns, one for users and one for memes, with checkboxes next to each result. If nothing matches, the page shows "no results available"

When the admin checks a user and a meme, the code adds that relationship to the UserMemes table. If the relationship already exists, it toggles it off instead of making a duplicate. This lets the

table. If the relationship already exists, it toggles it on instead of making a duplicate. This lets the admin add or remove user meme associations from the same page.



Saved: 5/7/2026 9:26:07 PM

Section #3: (1 pt.) Misc

Progress: 100%

☰ Task #1 (0.33 pts.) - GitHub Details

Progress: 100%

📁 Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history

Create current search query, filters, and sort

	Author	Editor	Proposer	Maintainer	Reviewer	Assignee	Tag
🔍 7 Open 42 Closed							
🔍 - admin meme assignment page							
#51 by Joshua-Emman (Owner) was merged 21 minutes ago							
🔍 - added unassociated meme admin page							
#50 by Joshua-Emman (Owner) was merged 1 hour ago							
🔍 + Add admin user meme association management							
#49 by Joshua-Emman (Owner) was merged 15 hours ago							
🔍 - Fix saved meme association table and save action							
#48 by Joshua-Emman (Owner) was merged 15 hours ago							
🔍 - Add saved meme list and remove actions							
#46 by Joshua-Emman (Owner) was merged 15 hours ago							
🔍 - fixed save button							
#44 by Joshua-Emman (Owner) was merged 16 hours ago							
🔍 + added usermeme table and save_meme.php							
#43 by Joshua-Emman (Owner) was merged 16 hours ago							

commit history



Saved: 5/7/2026 9:28:21 PM

🔗 Part 2:

Progress: 100%

Details:

Include the link to the pull request for Milestone3 to qa (should end in `/pull/#`)

URL #1

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/54>



URL

<https://github.com/Joshua-Emman>



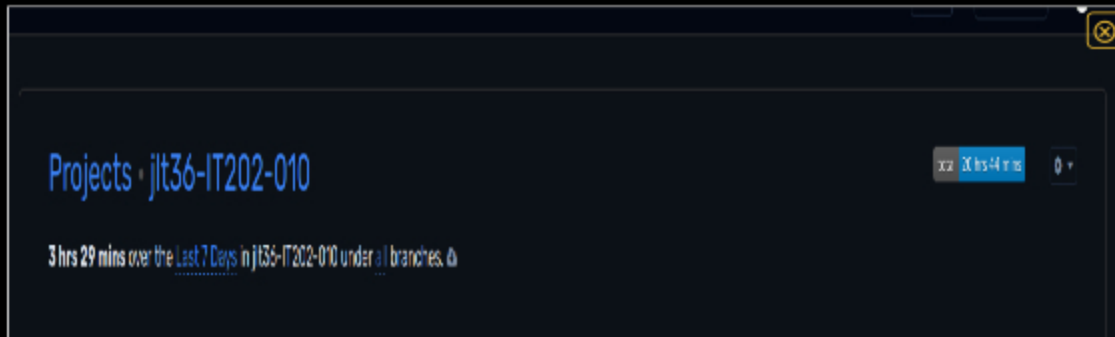
Saved: 5/7/2026 9:28:21 PM

📁 Task #2 (0.33 pts.) - WakaTime - Activity

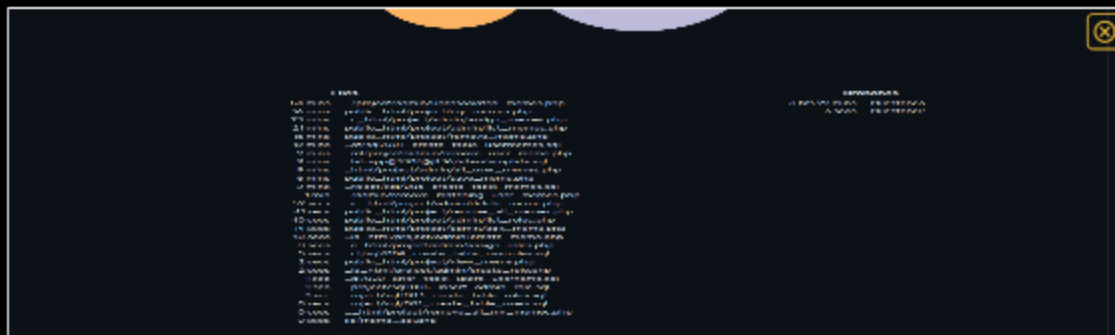
Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



overall time



file time

 Saved: 5/7/2026 9:40:04 PM

☰ Task #3 (0.33 pts.) - Reflection

Progress: 100%

☰ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

For this milestone, I learned how to connect my API data to users instead of just showing the data by itself. For my project, that meant making a UserMemes table so users could save memes to their account. I also learned how association

so users could save memes to their account. I also learned how association pages work, like showing one user's saved memes, showing all users' saved memes, and showing memes that are not saved by anyone yet. Another thing I learned is that deleting an association is different from deleting the actual meme. So the meme can stay in the database, but the user's saved connection to it can be removed.



Saved: 5/7/2026 9:42:11 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was making the smaller action pages, like removing one saved meme or removing all saved memes. Once I understood the pattern, those files were pretty quick to make because they all followed the same basic steps: check the user/admin, get the needed id, update the UserMemes table, show a flash message, and redirect back.



Saved: 5/7/2026 9:43:15 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was understanding how many different association pages were needed and how they were different from each other. Saving memes to one user seemed not too hard, but then I also had to make pages for all users' associations, unassociated memes, and admin assignment. Another hard part was getting the database queries right, especially the joins between Users, Memes, and UserMemes. I had to make sure removing an association only removed the relationship and did not delete the actual user or meme. The filters, stats, and limit requirements also made the pages take more time because each page needed to show the right results based on the current

search.



Saved: 5/7/2026 9:45:26 PM